



FORMAL METHODS REPORT

Lido

SRv3 Accounting Proof Closure Specimen

Specimen of a proof-closure report for Lido's SRv3 economic accounting state machine. The selected PO properties show the intended final claim, evidence, and trust-boundary shape for machine-checked Lean proofs over a focused Verity model.

Version	Private Specimen v1.1
Date	June 2026
Commit	PR-1811 / d088bbc2
Reviewers	LFG Labs

Mathematical proofs of correctness for smart-contract state machines.

lfglabs.dev

Confidentiality and result status

This document is a **private final-report specimen** for the selected SRv3 accounting proof-closure lane. It shows the claim structure, reader journey, and evidence layout that a final report would use once the companion Lean/Verity artifacts exist for PR #1811 at the commit named on the cover.

The report is intentionally narrow. It covers the economic state machine modeled for deposits, reserves, module balances, accepted reports, rewards, fees, and status gates. It does not claim full deployed-system equivalence, oracle truthfulness, cryptographic correctness, governance correctness, or exact proxy/storage refinement beyond the named assumptions.

In a production delivery, every specimen target row would be backed by a theorem file, a source-code cross-reference, a trust-boundary entry, and a rebuild command. In this specimen, those rows demonstrate the intended final shape rather than independently verifiable proof artifacts in this repository.

Distribution

Private review channel for Lido protocol contributors and selected technical reviewers. Public release, forum excerpts, or website publication should use strictly weaker language unless the proof repository and evidence manifest are available for independent rebuild.

Contents

Confidentiality and result status	i
---	---

DECISION SUMMARY

1 Executive summary	1
-------------------------------	---

PROOF RESULT

2 What the proof closes	4
3 Trust boundary	8

DELIVERY RECORD

4 Reproducibility and handoff	10
---	----

APPENDICES

A Appendix A: Reader glossary	12
B Appendix B: Certora delta	13
C Appendix C: Expected axiom and assumption summary	14
D Appendix D: Follow-on lanes	15

1 Executive summary

Question answered by this report

If PR #1811 changes how Lido accounts for deposits, reserves, module balances, and rewards, can the selected accounting rules be stated precisely enough that a machine can check them for every modeled case, not only for examples? This specimen shows the intended final-report answer once the companion proof repository exists: the answer would be yes for the six selected P0 (highest-priority) properties, under the named assumptions in Section 3.

In a final report, “closed” has a specific meaning: the theorem compiles without admitted proof obligations, its Solidity-facing model target is listed, all premises not discharged inside the model are registered by assumption ID, and the reproducibility command rechecks the theorem namespace from a clean checkout. Because this repository is a specimen, that sentence is the evidence threshold for a real delivery rather than a claim that the proof repository is present here.

The story in one page

The core safety question is simple enough to ask without Solidity or Lean: *when ETH and validator balances move through SRv3 accounting, can any value be spent from the wrong place, counted twice, rewarded from stale data, or sent through a disabled path?* The report answers that question by turning each informal concern into a small conservation law.

A useful mental model is three ledgers that must agree:

1. The **buffer ledger** says which ETH is depositable and which ETH is reserved for withdrawals.
2. The **router ledger** says how much ETH is pulled for deposits and how validator balances are summed across staking modules.
3. The **reward ledger** says which accepted balances may be used to compute rewards, fees, and recipients.

An “accepted report” means oracle-provided module-balance data that has passed the modeled SRv3 validation checks and is therefore allowed to update accounting state. The proof lane checks that PR #1811 preserves the three ledgers through those modeled transitions. It is not trying to prove every operational fact about deployment, cryptography, governance, gas, or the consensus layer.

Why this is more than testing

A test asks, “does this example work?” A theorem asks, “does this rule survive every modeled case that satisfies the assumptions?” That is why the report spends so much effort naming the model and assumptions: they define the world in which the machine-checked statement is meant to be universal.

The final claim this specimen is designed to support is:

For all modeled finite module arrays and accepted report arrays satisfying the listed well-formedness assumptions, the selected PR #1811 SRv3 economic state machine preserves deposit conservation, reserve separation, module-balance conservation, report-before-reward consistency, reward/fee bounds, and flow-specific status gating.

This would not be a claim that every deployed-system behavior is proved. The exact boundary is part of the result: cryptographic witnesses, consensus truthfulness, governance configuration, and deployment refinement are named in Section 3.

How to read the proof

Read each row below as a three-step promise for the final package. First, the Solidity-facing model names the transition being studied. Second, the property states what must never go wrong for any modeled input. Third, Lean should check the resulting theorem against the Verity-generated state model and the explicit assumptions. The value of the exercise is not that one test passed; it is that the statement leaves no remaining case split for a reviewer to hold in their head.

The matrix answers six plain questions: what ETH is spendable, what deposits consume, what balances add up to, what report rewards use, how fees are bounded, and which module states block economic flow.

Proof matrix

All six rows below are specimen targets: they show the intended final closed state once the corresponding theorem artifact and trust-boundary row are present in the companion proof repository.

ID	PR #1811 surface	Intended final property
SRV3-P1	Lido buffer allocation	Withdrawal-reserved ETH is excluded from the depositable resource, so deposit spending cannot borrow from withdrawal liquidity.
SRV3-P2	Router deposit path	Router-pulled ETH equals 32 ETH times actual deposits and is fully sunk into CL deposits, with no modeled residual held by the router.
SRV3-P3	Module balances	Router-wide validator balance is exactly the sum of accepted per-module balances, so the aggregate cannot drift away from the module ledger.
SRV3-P4	Oracle report ordering	Module fee-distribution weights come from the current accepted module-balance state, not from stale or partially applied reports.
SRV3-P5	Rewards and fees	Module fee-distribution weights and recipient alignment use accepted module balances; total reward and fee magnitude remain accepted report or vault-context inputs under assumptions.
SRV3-P6	Status gates	Non-active or deposit-paused modules receive no modeled deposit allocation; stopped modules receive no module-side reward or fee allocation.

Review posture

This specimen is written for two readers at once. A protocol reviewer can follow the ledgers and the trust boundary without reading Lean. A formal-methods reviewer can

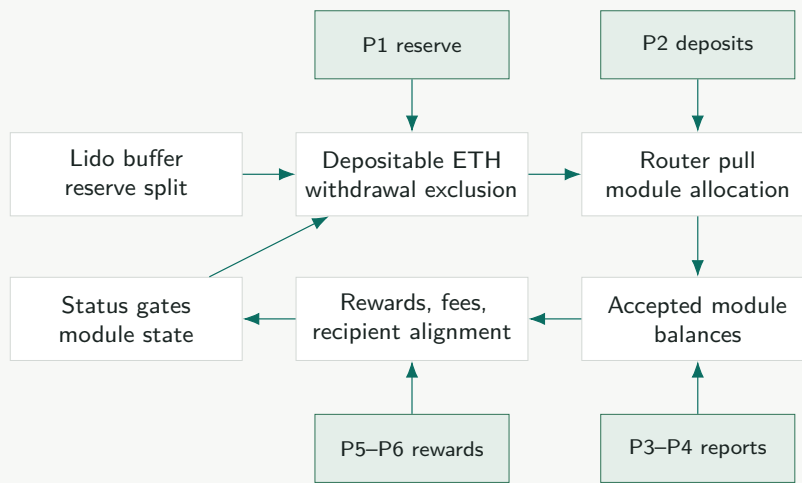
check whether the theorem IDs, assumptions, and rebuild gates would support the advertised claim in a real delivery package.

2 What the proof closes

A proof is closed when the theorem checker accepts it without admitted proof steps and the remaining premises are named as assumptions. This section shows the six accounting claims in three forms: a system map, a plain-language story, and specimen theorem rows for the final evidence package.

Modeled architecture

The proof model follows the accounting path where PR #1811 concentrates fund-safety risk. In plain terms, the model follows ETH as it moves from the Lido buffer, through the router, into staking-module deposits, and then into later balance reports and reward calculations. Each proof asks whether that flow preserves the accounting rule it is supposed to preserve. External contracts are summarized only at the interfaces used by those transitions.



From intuition to theorem

Read the six properties as one accounting story. P1 protects ETH reserved for withdrawals. P2 checks that initial deposits pull exactly the ETH they should. P3 checks that the router's total balance equals the module balances it accepted. P4 makes module fee weights depend on the accepted report, not an old or partial one. P5 bounds reward and fee movement. P6 blocks non-active or paused modules from deposit allocation and stopped modules from module-side reward or fee allocation.

Each invariant below has two layers. The first sentence says why the property matters in ordinary accounting language. The formula then states the same idea in a form that can be checked over all modeled inputs. You can read the formulas like receipts: they name the money bucket before and after a move, then check that the totals still match. The symbols below only stand for modules, balances, reserves, and reports already described in the ledger story.

Let M be the finite set of staking modules in router order. For a module $m \in M$, let bal_m be its accepted validator balance in gwei, status_m its module status, and moduleRecipient_m its module reward recipient in the modeled reward step. Let B_R be the router validator balance, T the buffered ETH, R_D the stored deposit reserve, and R_W the withdrawal-reserved buffer. The theorems are parametric in the finite module count, while each accepted report must still cover the current router module set in router order, with unique

module IDs, matching array lengths, and the freshness and range premises named in the theorem signature.

P1: reserve separation.

$$d = \min(T, R_D) \quad w = \min(T - d, R_W) \quad u = T - d - w$$

$$\text{deposable}(T) = d + u$$

$$\forall a. \text{ spendDeposit}(a) \Rightarrow a \leq \text{deposable}(T)$$

$$\text{spendDeposit}(a) \Rightarrow \text{usesWithdrawalReserve}(a, T, R_D, R_W) = 0$$

Plain reading: some ETH is saved for withdrawals, and deposits are not allowed to spend that saved amount. Deposit spend is bounded by the pre-spend deposit reserve plus unreserved allocation and does not consume the pre-spend effective withdrawal-reserved bucket. The theorem sketch deliberately avoids a raw post-state equality for R_W ; any post-state reserve fact belongs to the model's explicit reserve-recomputation rule.

P2: exact router pull.

$$\text{pulledWei} = 32 \text{ ETH} \cdot \text{actualDeposits} \quad \text{routerEthAfter} = \text{routerEthBefore}$$

$$\text{actualDeposits} = \sum_{m \in M} \text{allocatedDeposits}_m$$

Plain reading: across the modeled atomic initial-deposit transition, the router pulls exactly one 32 ETH unit per validator and immediately forwards it to the beacon deposit sink. Top-ups, if included in a later lane, are a separate gwei-aligned allocation theorem and are not instances of this 32 ETH exact-pull statement.

P3: module-balance conservation.

$$\text{acceptedReport}(M, \vec{b}) \Rightarrow B'_R = \sum_{m \in M} \text{bal}'_m = \sum_{i=0}^{|M|-1} \vec{b}_i$$

Plain reading: the router total must equal the sum of the module rows after an accepted report. The proof is over arbitrary finite module arrays, with module-ID uniqueness and report alignment named as assumptions rather than hidden bounds.

P4: report-before-reward consistency.

$$\text{rewardStep}(s, s') \Rightarrow \exists r. \text{acceptedAt}(r, s) \wedge \text{rewardBase}(s) = \text{balances}(r)$$

Plain reading: accepted module balances determine the module fee-distribution weights used by the modeled reward step. Total reward magnitude and protocol fee inputs are treated as accepted report or vault-context inputs subject to the listed range and freshness assumptions. In simpler terms, rewards should be split using the report the system just accepted, not a previous snapshot and not a half-updated table.

P5: bounded rewards and fee alignment.

$$0 \leq \text{moduleFee}_m \leq \text{totalReward} \cdot \text{moduleFeeBps}_m / 10,000$$

$$\text{moduleRecipient}'_m = \text{moduleRecipient}_m \wedge \text{rewardRecipient}(\text{moduleFee}_m) = \text{moduleRecipient}_m$$

Plain reading: fee amounts cannot exceed the configured basis-point bound, and each amount remains paired with the recipient for the module that earned it.

P6: status gating.

$$\text{status}_m \neq \text{active} \vee \text{depositsPaused}_m \Rightarrow \text{allocatedDeposits}_m = 0$$

$$\text{status}_m = \text{stopped} \Rightarrow \text{moduleReward}_m = 0$$

Plain reading: a stopped or paused module is not just marked differently in storage; that status changes the allowed flow. Deposit-paused or non-active modules receive zero modeled deposit allocation. Stopped modules receive zero module-side fee or reward allocation. Any residual protocol-fee treatment is stated separately from module recipient payment.

Specimen traceability register

The next table is for auditability. The theorem name is the handle a reviewer would search for in Lean. The expected artifact says where the proof should live. The assumption IDs say which facts the proof is allowed to borrow from outside the model. A final report should not ask the reader to trust prose alone; every claim should have this chain.

ID	Proof handle	Assumptions	Claim
SRV3-P1	<code>Reserves.lean</code> <code>reserve</code> <code>separation</code>	A-EXT-01; A-ARITH-05	Deposit spend cannot consume withdrawal-reserved ETH.
SRV3-P2	<code>Deposits.lean</code> <code>deposit_exact</code> <code>pull</code>	A-DEP-02; A-ID-04; A-ARITH-05	Pulled wei equals 32 ETH times actual deposits over all allocated modules.
SRV3-P3	<code>Balances.lean</code> <code>module_balance</code> <code>sum</code>	A-ORC-03; A-ID-04; A-ARITH-05	Accepted report application preserves router balance as the module sum.
SRV3-P4	<code>Rewards.lean</code> <code>reward_uses</code> <code>accepted_report</code>	A-ORC-03; A-ID-04	Module fee-distribution weights read the latest accepted module-balance state.
SRV3-P5	<code>Rewards.lean</code> <code>reward_fee</code> <code>bounded_aligned</code>	A-ORC-03; A-ARITH-05	Fee amounts are bounded and recipient/module arrays stay aligned.
SRV3-P6	<code>Status.lean</code> <code>status_gates</code> <code>economic_flows</code>	A-EXT-01; A-ID-04	Non-active or deposit-paused modules receive no modeled deposit allocation; stopped modules receive no module-side reward or fee allocation.

Evidence architecture

Layer	Artifact	Role
Solidity-facing summaries	<code>verity/SRv3</code>	Preserve the relevant array, loop, reserve, and call structure while summarizing external contracts through explicit interfaces.
Economic model	<code>State.lean</code>	Defines modules, reserves, Wei/Gwei arithmetic, status flags, accepted reports, rewards, and fee state.
Proof obligations	<code>Properties/*.lean</code>	States each target over arbitrary finite module/report arrays, with uniqueness, ordering, alignment, and freshness premises named in theorem signatures.
Trust-boundary report	<code>trust-boundary.json</code>	Machine-readable list of assumptions, theorem IDs, axiom output, source links, retained Certora assumptions, and out-of-model claims.
Rebuild gate	<code>lake build</code>	Rechecks all selected theorem files and fails if any trusted declaration, admitted theorem, or unmapped assumption appears outside the allowlist.

3 Trust boundary

A formal proof is not a proof of “everything the deployed system can ever do.” It is a proof about a specific model. In plain terms, this model checks the accounting moves inside the selected SRv3 state machine; it does not prove that every outside system feeding that state machine is honest, available, or cryptographically sound. The boundary below is therefore part of the result, not a footnote.

What is inside the model

Inside	Why it is modeled
Reserve and buffer arithmetic	This is where depositable ETH must be separated from withdrawal-reserved ETH.
Router deposit accounting	This is where pulled ETH must match actual 32 ETH validator deposits.
Module-balance reports	This is where per-module balances become the router-wide accepted balance.
Rewards, fees, and recipients	This is where amount bounds and recipient alignment matter.
Module status gates	This is where non-active or paused modules stop receiving modeled deposit allocation, and stopped modules stop receiving module-side rewards or fees.

Named assumptions

An assumption is not a hidden conclusion. It is a fact this proof is allowed to start from. The point of naming assumptions is to show exactly which parts still need ordinary audit, deployment review, oracle review, or cryptographic review.

ID	Assumption	Meaning
A-EXT-01	External contract summaries	Lido buffer withdrawals, staking module callbacks, fee recipients, and beacon deposit sinks conform to the specified interfaces and mutate no unlisted SRv3 accounting state.
A-DEP-02	Beacon deposit sink	A modeled beacon deposit removes exactly the intended 32 ETH value from the router-side resource. Deposit-root, BLS, and SSZ correctness are not proved.
A-ORC-03	Accepted oracle reports	Once accepted by SRv3 validation, module-balance reports satisfy the listed shape, alignment, freshness, and range assumptions. Consensus-layer truthfulness is outside this report.

ID	Assumption	Meaning
A-ID-04	Module identity and ordering well-formedness	Module IDs are unique, report arrays align with those IDs where required, and sortedness obligations are enforced for ordered paths.
A-ARITH-05	Unit-conversion domain	Wei/Gwei and basis-point arithmetic is interpreted over the bounded unsigned integer domains used by the modeled transitions.

Explicit non-claims

Boundary	Treatment
BLS signatures, deposit roots, SSZ, Merkle proofs, EIP-4788	Named cryptographic and consensus-data boundaries, not part of the P0 accounting closure.
Exact packed-storage migration and proxy mechanics	Deferred to a deployment and migration lane; not used as proof evidence here.
Full governance role graph and parameter admissibility	Configuration is assumed to satisfy the stated range and role assumptions; unauthorized governance transitions are not modeled.
CL witness soundness and VaultHub	Not proved in this lane. If their values enter an accounting transition, they enter only through named, well-formed input assumptions; otherwise no theorem relies on them.
Gas, events, revert strings, and exact call-stack shape	Engineering and integration review surfaces, not conservation-law premises.

Allowed public wording

Precise specimen claim

This report specimen demonstrates the intended final claim shape: the selected PR #1811 SRv3 accounting invariants would be reported as closed over a focused Lean/Verity economic-state-machine model, under explicit assumptions for external contracts, accepted oracle inputs, cryptographic witnesses, governance configuration, and deployed-system refinement. Use closure language publicly only after the companion proof repository and evidence manifest are available for independent rebuild.

4 Reproducibility and handoff

Reviewer command in a real delivery

A reviewer does not need to trust the PDF alone. The handoff includes the proof sources and commands that rebuild the selected theorem set and check that the PDF's assumptions match the machine-readable trust-boundary report.

The rebuild is the reviewer's receipt. If it succeeds, the reviewer learns that the theorem names in the PDF exist, the proof checker accepts them, and the listed assumptions match the machine-readable trust report. If it fails, the PDF claim is not ready to rely on.

```
git clone git@github.com:lfglabs-dev/lido-srv3-proof-closure.git
cd lido-srv3-proof-closure
lake build SRv3Proofs
python3 scripts/check_trust_report.py reports/trust-boundary.json
```

In a production delivery, this command checks all selected theorem files, verifies that every matrix row has a matching artifact, and rejects unstated trust assumptions in the machine-readable report. In this specimen, the command is a contract for the handoff package rather than an executable artifact shipped in this repository.

Minimum provenance fields

Field	Expected content
PR and source revision	PR #1811 URL, base branch, audited commit hash, and review timestamp. Specimen target: d088bbc2deac9913b68036d73d35c37aa6279b90.
Proof toolchain	Lean toolchain, Lake manifest, Verity commit, and model generation command.
Theorem namespace	Selected theorem modules, expected target names, and the command that fails on <code>sorry</code> , <code>admit</code> , custom axioms, or missing source links.
Trust allowlist	Machine-readable assumption IDs, out-of-model boundaries, and theorem-to-assumption mapping.
Artifact integrity	SHA-256 hashes for the PDF, proof manifest, trust report, and generated model artifacts.

Source anchor examples for the final manifest

The specimen matrix uses broad PR #1811 surfaces to stay readable. A production source map should expand those surfaces into exact file/function anchors. Examples for the target source revision include:

- Buffer and reserve accounting: `Lido.sol::_getBufferedEtherAllocation`, `getDepositAbleEther`, and `withdrawDepositAbleEther`.
- Deposit allocation: `StakingRouter.sol::deposit` and `SRLib.sol::_getDepositAllocations`.
- Reports and rewards: `StakingRouter.sol::reportValidatorBalancesByStakingModule`, `getStakingRewardsDistribution`, and `SRLib.sol::_reportRewardsMinted`.
- Oracle validation: `AccountingOracle.sol::_checkStakingRouterModuleBalances`.

Handoff contents

- **PDF:** `dist/lido-srv3-formal-methods-report.pdf`. This report specimen for Lido reviewers.
- **Certora map:** `certora-pr1811-map.md`. Mapping from Certora V2/V3 specs and assumptions to PR #1811 proof targets.
- **Trust report:** `reports/trust-boundary.json`. Machine-readable theorem, assumption, and boundary register.
- **Proof sources:** `lean/` and `verity/`. Source artifacts for proof review and future extension.
- **Traces:** `counterexamples/`. Empty for the selected P0 set; retained for future falsified targets or regression traces.

Integrity checks

Check	Expected result
<code>lake build SRv3Proofs</code>	All theorem modules compile with no admitted proof obligations.
<code>check_trust_report.py</code>	Every theorem, assumption, and declared source anchor in the final evidence manifest has a machine-readable counterpart.
<code>grep -R "sorry admit"</code>	No admitted Lean proofs in the selected theorem namespace.
<code>scripts/replay_verity.sh</code>	Regenerates the Verity summaries used by the Lean state model.

A Appendix A: Reader glossary

Router The SRv3 component that allocates deposit work across staking modules and records aggregate module accounting.

Module A staking-provider lane managed by the router.

Accepted report Oracle balance data that has passed the modeled SRv3 validation checks and may update accounting state.

Withdrawal reserve ETH kept aside for withdrawals, not available for new deposits.

CL Consensus layer, the Ethereum system that records validator activity.

Lean The proof checker that verifies theorem statements.

Verity The translation and modeling layer used to connect Solidity-facing structure to Lean proofs.

P0 Highest-priority property class. Here it means an accounting claim important enough that a final delivery should either prove it or clearly say why it remains open.

Theorem A precise statement that Lean checks. In this report, theorem names are handles for tracing prose claims back to proof files.

Assumption A named premise that the theorem may use but does not prove.

Trust boundary The edge of the proof: what the model proves, what it assumes, and what remains for separate review.

B Appendix B: Certora delta

This lane does not replace earlier Certora work. It shows how a PR #1811 proof-closure package would continue from that work: moving from bounded or summarized checks toward explicit SRv3 accounting transitions, balance-based module reasoning, and named assumptions that a reviewer can audit.

Legacy coverage class	Disposition	Final treatment
V2 validator-count accounting	Continued	Reframed as module-balance conservation over accepted per-module balances.
V2 deposit availability bounds	Strengthened in scope	Expressed as reserve separation plus exact 32 ETH pull accounting.
V2/V3 reward distribution checks	Strengthened in scope	Expressed as accepted-balance reward computation, fee bounds, and recipient alignment.
Router status-gating assertions	Reused and extended	Applied to active, stopped, and deposit-paused gates in the modeled economic paths.
V3 bounded module-array specs	Generalized	Theorems are stated over arbitrary finite arrays with named well-formedness assumptions.

C Appendix C: Expected axiom and assumption summary

Class	Expected final value	Interpretation
Lean axioms introduced by LFG	0	The final selected theorem namespace should contain no custom axioms.
Admitted theorem bodies	0	The final selected theorem namespace should contain no <code>sorry</code> or <code>admit</code> .
Named model assumptions	5	The specimen assumes this allowance: external summaries, beacon sink behavior, accepted oracle reports, module identity/order well-formedness, and arithmetic domain.
Selected falsification/regression harness counterexamples	0	A final package should report no selected P0 counterexample trace in the named harness. This is not a claim about unmodeled deployed-system behavior.

D Appendix D: Follow-on lanes

These lanes are recommended because they touch adjacent SRv3 risk surfaces, not because they are required to support the accounting claims in this specimen. Recommended sequence after this P0 accounting closure:

1. ExitBus authorization, replay resistance, duplicate prevention, and triggerable-exit fee/refund exactness.
2. Consolidation fee conservation, source and target binding, and source-ownership caveats.
3. Top-up and consensus-layer proof binding.
4. Oracle sanity limits, drain guards, and second-opinion oracle composition.
5. Deployment governance, role graph, and exact migration refinement.
6. VaultHub F-01/F-02/F-03, if still relevant to the active review path.